TU WIEN Informatics

Master Thesis Proposal

An Agent-Based Architecture for Automated MDA Transformations from Heterogeneous Enterprise Models to Low-Code Platforms

Candidate:
Thorben Lövvelt
12402552

Supervisor: Univ. Prof. Henderik Proper, PhD

1 Motivation and Problem Statement

1.1 Motivation

Model-Driven Architecture (MDA), standardised by the Object Management Group (OMG) [17], aims to structure software development through systematic transformations between levels of abstraction. The approach distinguishes between the following three model levels: The Computation-Independent Model (CIM) describes domain concepts and business requirements without technical details. The Platform-Independent Model (PIM) specifies the system functionality independently of specific technologies. Finally, the Platform-Specific Model (PSM) combines the PIM specification with platform-specific implementation details. Languages such as ATL [9] and QVT [16] have become established for the transformation between these levels.

The combination of MDA with low-code platforms, which enable visual application development with minimal coding effort, is particularly promising [20]. Platforms such as Mendix offer public APIs for programmatic model creation [13], making end-to-end transformation pipelines technically feasible. This approach has already been investigated in several studies. Krouwel et al. [11] have implemented an automated transformation from DEMO to Mendix using TypeScript and JSON as an intermediate format. Their implementation is publicly available and has been validated in several case studies. Bzowski [4] made an important conceptual contribution in his diploma thesis at TU Wien. He developed pimUML as an explicit platform-independent model and formalised transformation rules according to MDA principles. His work defines theoretical mapping rules from DEMO Fact Models via pimUML to Mendix, which he manually demonstrated and evaluated using a car-rental case study. He identified the automation of these transformations as future work.

In practice, however, CIM models are created using different tools and languages. Organisations choose their modelling language based on methodological preferences and domain-specific requirements. Two prominent examples are, for instance, DEMO [6], which is based on the Enterprise Ontology and describes enterprises in a fact-oriented manner and MERODE [21], which focuses on existence dependencies between business objects and their lifecycles. This heterogeneity creates a fundamental challenge. Each CIM tool produces models in its own format, whilst each target platform expects input in its specific format.

What is missing is an open middle layer capable of accepting various CIM formats and processing them in a uniform manner. Blanc et al. [2] pursued a similar idea with their Model Bus for the interoperability of UML tools, but without reference to ontological enterprise modelling or low-code platforms. Previous work, including the automated implementation by Krouwel et al. [11] and the theoretical transformation rules by Bzowski [4], each support only DEMO as an input language. If one wishes to integrate another language, the entire transformation chain must be redeveloped. There is no mechanism that allows new input languages to be added without modifying existing components.

1.2 Problem Statement

Existing MDA transformation approaches for mapping enterprise models to low-code platforms are limited to specific modelling languages and have a monolithic structure [4, 11]. This heterogeneity manifests at two levels: at the *language level*, CIM notations differ fundamentally in their paradigms and at the *tooling level*, each language comes with its own modelling environment and proprietary export format. There is therefore a lack of an extensible architecture capable of processing heterogeneous CIM languages and their associated tool outputs via a common, schema-validated intermediate format and which enables the integration of new input languages simply by adding a transformation agent, without having to modify all existing components. As a concrete indicator of this limitation, all known automated transforma-

tion systems support exactly one CIM input language [4, 11], requiring a full reimplementation of the transformation chain for each additional language.

1.3 Solution Approach

This thesis proposes an agent-based reference architecture, hereinafter referred to as the *Modelling Bus*, that decouples the transformation logic for each CIM language. Each agent is responsible for exactly one input language and must possess two capabilities: parsing its specific input format and generating output according to a common PIM schema. The conceptual basis for this PIM is provided by the work of Bzowski [4], whose pimUML profile is used as the starting point for defining a JSON Schema-based intermediate format. The automation that Bzowski identified as 'Future Work' is also addressed in this thesis. Furthermore, the concept is extended to heterogeneous input languages. Supporting an additional language thus requires only the implementation of a new agent. All downstream components for PIM validation, PSM generation and platform export should therefore remain unchanged.

2 Goals and Expected Outcome

2.1 Research Questions

The research questions are structured following Wieringa [24], who distinguishes between design problems and knowledge questions and draw on the DSR question typology of Thuan et al. [23]. Together, these frameworks suggest organising research questions across three orientations: understanding what is already known, framing what needs to be built and asking how it can be built and evaluated.

RQ1 *What is the current state of MDA transformation approaches for enterprise modelling languages and what architectural and conceptual gaps prevent existing approaches from supporting heterogeneous CIM input languages?*

This question is addressed through a Systematic Mapping Study [10] and provides the empirical foundation for the subsequent design problem.

RQ2 *Which requirements must a model transformation architecture fulfil to support heterogeneous CIM input languages without modifying existing components and which structural properties must the Platform-Independent Model (PIM) satisfy to serve as a common transformation target for paradigmatically different languages such as DEMO [6] and MERODE [21]?*

Following Wieringa [24], artifact design should be grounded in explicit requirements before construction begins. This question therefore precedes the architectural design and defines the criteria against which the Modelling Bus will later be evaluated.

RQ3 *What architectural decisions are required to realise a loosely coupled, plugin-based Modelling Bus in which each CIM agent independently transforms its input into a JSON Schema-validated PIM and does the resulting architecture support automated end-to-end transformation to a low-code platform such as Mendix without modification of existing components?*

This question is answered if a new CIM agent can be integrated without touching any existing component, as operationalised by the extensibility metric in Section 2.3. The prototype implementation (D5) provides the empirical basis for this assessment.

RQ4 *To what extent does the implemented Modelling Bus satisfy the requirements identified in RQ2 across heterogeneous input languages and which dimensions of semantic correctness, extensibility and traceability are preserved in the transformation?*

2.2 Expected Deliverables

By the end of this thesis, the following deliverables should be available: A requirements specification for extensible transformation systems, building on the results of the preceding systematic mapping study (**D1**). A reference architecture for the Modelling Bus with documented design decisions (**D2**). JSON Schema specifications for all relevant abstraction levels, for both CIM and PIM (**D3**). Formalised transformation rules from DEMO Fact Model to PIM and from MERODE to PIM (**D4**). A prototype implementation with functioning agents for both languages, as well as Mendix export (**D5**). An evaluation framework with operationalised metrics (**D6**). Empirical results from the application to the car rental case study (**D7**).

2.3 Success Criteria

The evaluation is based on the following criteria. The specific thresholds are derived methodologically from the relevant literature, in particular from the Semiotic Quality Framework [12] and the Quality Framework for Conceptual Models [14] and operationalised within the evaluation framework (D6).

The system must support at least two CIM languages, specifically the DEMO Fact Model and MERODE, as demonstrated by a functional demonstration of the transformation. *Semantic completeness* is verified and indicates what proportion of the statements from the source model is retained in the target model [12, 14]. *Semantic Validity* checks the correctness of the transferred statements [12, 14]. A distinction is made between missing and incorrectly transformed elements, as the latter can have more serious consequences in the context of automated pipelines. The *Schema Validation Rate* must achieve full compliance to ensure the structural integrity of the intermediate model. *Extensibility*, defined in accordance with the quality characteristic of modifiability as per ISO/IEC 25010 [8], requires that a new transformation agent can be integrated without changes to existing components. The architectural implementation is based on the principles of loosely coupled systems [15]. The *Traceability Coverage* measures the proportion of source elements with a documented trace link to the output model. In this thesis, the metric is defined on the basis of the traceability dimensions identified by Winkler and Pilgrim [25]. A tolerance is provided for technical auxiliary structures without a direct semantic counterpart in the source model. The target values for these metrics will be derived from the literature and operationalised in the evaluation framework (D6) once the transformation rules are formalised. As a general orientation, higher completeness and validity scores indicate fewer semantic losses and errors in the pipeline, whilst a Schema Validation Rate below 100% would indicate structural integrity issues that require correction before the pipeline can be considered functional. Extensibility is treated as a binary criterion: the integration of a new agent either requires changes to existing components or it does not.

3 Methodological Approach

3.1 Research Methodology

This work follows the Design Science Research paradigm as outlined by Hevner et al. [7] and Peffers et al. [18] and adopts the distinction between design problems and knowledge questions as introduced by Wieringa [24]. The DSR question typology of Thuan et al.[23] further informs the structure of the research questions across the orientations of knowing, framing and designing. The primary objective is the development of a constructive artefact, the Modelling Bus, which encompasses architecture, a schema-based intermediate format, formalised transformation rules and a prototypical implementation. Evaluation is carried out through demonstration using a case study, as well as systematic quality analysis

based on defined metrics. The intended contribution is both practical, in the form of a working system and theoretical, in the form of transferable architectural principles and formalised transformation rules.

3.2 Research Approach

The research approach is divided into eight sequential steps. The first step involves *Problem Identification* through literature analysis and gap analysis using a Systematic Mapping Study [10]. Given the narrow and emerging nature of the field, a mapping study is more appropriate than a full Systematic Literature Review, as it provides a structured overview of existing approaches without requiring exhaustive meta-analysis. The second step comprises *Requirements Definition* by deriving requirements from the literature and existing systems according to Pohl [19], resulting in the requirements catalogue (D1). The third step is the *Architecture Design* of the Modelling Bus, in which the requirements form D1 are translated into architectural descisions following the quality-attribute-driven approach of Bass et al. [1] (D2). The fourth step deals with the *Schema Design* through analysis of the DEMO and MERODE metamodels and the identification of common abstractions, using JSON Schema as a meta-language in line with Colantoni et al. [5] (D3). The fifth step involves *Mapping Development*. The transformation rules are formalised based on the QVT concepts of the OMG [16], whereby explicit mappings between source metamodel elements and PIM schema elements are defined for each CIM language (D4). The sixth step involves the *implementation* of the prototype, which operationalises the designed artefact as a part of the DSR build cycle [7], resulting in (D5). The seventh step involves the *demonstration* through application to the Rent-A-Car case study, following the case study method of Yin [26]. The eighth step deals with the *evaluation* through multi-dimensional quality analysis based on the Semiotic Quality Framework [12] and the Quality framework for Conceptual Models [14] (D6, D7).

3.3 Evaluation Methodology

Semantic correctness is evaluated on the basis of the Semiotic Quality Framework [12] and the Quality Framework for Conceptual Models [14]. The procedure is divided into three phases. In the ground truth extraction phase, all atomic statements are extracted from the source model by verbalising the model elements [6]. For DEMO Fact Models, this includes entity types, fact types, property types, cardinalities and derivation rules. For MERODE, these are object types, existence dependencies, business event types and finite state machines. In the transformation execution phase, the automated pipeline is run with traceability logging. In Output Verification, each source fact is classified as Correct, Missing, Incorrect or Added. This four-way classification extends the binary distinction proposed by Lindland et al. [12] to include the category Incorrect, in order to differentiate between semantically incorrectly transformed elements and structurally added elements. The resulting metrics are Completeness (proportion of correctly retained statements) and Validity (absence of incorrect or unjustifiably added statements in the target model). Beyond semantic correctness, the evaluation also covers extensibility and traceability as defined in Section 2.3. Extensibility is tested by checking whether a third CIM agent could be added without touching any existing component, in line with ISO/IEC 25010 [8]. Traceability Coverage verifies that each source element has a corresponding trace link in the output, following Winkler and Pilgrim [25]. Both dimensions are part of the quality assessment for RQ4. The prototype implementation (D5) serves as the central validation instrument. All metrics defined above are measured against the output of the running system, ensuring that the evaluation assesses not only conceptual soundness but also technical feasibility under realistic conditions.

4 State of the Art

4.1 Model-Driven Architecture

Model-Driven Architecture is standardised by the OMG [17] and structures software development through transformations between abstraction levels, from CIM via PIM to PSM. Languages such as ATL [9] and QVT [16] have become established for the formalisation of such transformations. However, a key challenge remains the interoperability between tools operating at different levels of abstraction. Blanc et al. [2] addressed this with their Model Bus, a service bus approach for the interoperability of UML-based CASE tools. However, their approach remains limited to UML and does not consider either ontological enterprise modelling or low-code platforms as target levels.

4.2 Enterprise Modeling Languages

At the CIM level, there are various modelling languages that describe enterprises from different paradigmatic perspectives. DEMO [6] is based on PSI theory and Enterprise Ontology and follows a fact-oriented approach. The Fact Model relevant here describes the static-structural aspects of a domain through Entity Types, Fact Types, Property Types and Derivation Rules. MERODE [21] takes a complementary approach. It models existence dependencies between business objects and their lifecycles using business object types, existence dependencies, business event types and finite state machines. Both languages describe structural aspects using fundamentally different paradigms. The DEMO Fact Model is fact-oriented and MERODE is object-lifecycle-oriented, making them ideal candidates for validating a language-independent architecture.

4.3 Low-Code Development Platforms

Low-code platforms enable the visual development of applications with minimal coding effort [3, 20]. Unlike traditional code generators, platforms such as Mendix are themselves model-based. Applications are stored as model instances, consisting of a domain model for the data structure, microflows and nanoflows for the business logic and pages for the user interface [13]. This model-based structure makes Mendix conceptually particularly suitable as a PSM target level in an MDA pipeline, as the transformation takes place not in code but in platform models. Mendix also offers a public API for programmatic model creation [13], making automated transformation pipelines technically feasible.

4.4 Intermediate Representations and Transformation Architectures

The choice of intermediate format is a key design decision in any MDA pipeline. Traditionally, MOF-based metamodels and their serialisation in XMI have dominated the MDA tool stack, supported by frameworks such as the Eclipse Modeling Framework (EMF) with its Ecore metamodel [22]. These formats offer strong expressiveness and tool support within Eclipse-based toolchains, but are closely tied to specific runtime environments. JSON Schema takes an alternative approach. It enables the structural validation of model data in a format that can be natively integrated into web APIs, low-code platforms and CI/CD pipelines. Colantoni et al. [5] demonstrate that JSON Schema can be used as a meta-language for specifying structural constraints in the MDE context and bridge the gap between the JSON and modelling technology domains. In the context of this work, JSON Schema is chosen as the PIM format, as the target platform, Mendix, itself operates on a JSON basis and a consistent JSON-based pipeline minimises the technology gap between PIM and PSM.

At the architectural level, the question arises as to how heterogeneous input formats can be integrated into a unified pipeline. The Model Bus proposed by Blanc et al. [2] adopted a service bus approach for UML tools, in which tools communicate via standardised interfaces. This principle of loose coupling via a central bus is related to the Mediator pattern and plugin architectures, as established in software engineering for extensible systems [15]. This thesis applies this principle to enterprise modelling. Each CIM agent functions as a standalone plugin that docks onto the Modelling Bus via a defined PIM schema.

4.5 Existing Transformation Approaches

The combination of enterprise modelling and low-code has so far been investigated in two complementary studies. Krouwel et al. [11] present an automated end-to-end transformation from DEMO to Mendix using TypeScript with JSON as an intermediate format. Their implementation is available on GitHub and has been evaluated in several case studies (EU-Rent, Volley, Social Housing). However, an explicit formal PIM layer in accordance with the MDA specification is missing. Furthermore, the architecture is monolithic and designed with DEMO as the sole input language. Bzowski [4] addresses the missing PIM layer and develops pimUML as an explicit UML profile for the Platform-Independent Model between DEMO and Mendix. He formalises transformation rules and evaluates semantic correctness using the Rent-A-Car case study. However, the transformations were demonstrated manually. Bzowski explicitly identifies automation as future work.

4.6 Research Gap

The research gap can be summarised on two levels. At the conceptual level, there is a lack of an architectural approach that decouples the MDA transformation chain from the specific input language, thereby opening it up in principle to heterogeneous CIM languages. At the technical level, there is no implementation that combines a formally defined PIM with automated end-to-end transformation whilst ensuring extensibility to new input languages without modifying existing components. Furthermore, the quality evaluation of previous approaches is limited to individual dimensions, whilst a multi-dimensional assessment incorporating validity, traceability and extensibility remains to be carried out. The focus on the DEMO Fact Model enables a clear comparison with MERODE. An extension to further DEMO aspects is Future Work.

5 Relevance to the Curriculum

This thesis draws on four areas of knowledge that are each addressed in the Master's programme in Business Informatics. Understanding DEMO and MERODE well enough to formalise transformation rules requires a solid grounding in enterprise modelling, which is built in *Enterprise & Process Engineering* through a comparative analysis of both languages. The technical core of the thesis, metamodelling, model transformations and code generation, relies on model-driven engineering expertise covered in *Model Engineering*, with topics such as ontology-driven modelling and web-based modelling environments addressed in *Advanced Model Engineering*. Finally, designing the Modelling Bus as a loosely coupled, plugin-based system requires familiarity with service-oriented and distributed architectures, which is the focus of *Advanced Internet Computing*.

References

[1] Len Bass, Paul Clements, and Rick Kazman. *Software Architecture in Practice*. 3rd. Upper Saddle River, NJ: Addison-Wesley, 2013.

[2] Xavier Blanc, Marie-Pierre Gervais, and Prawee Sriplakich. "Model Bus: Towards the Interoperability of Modelling Tools". In: *Model Driven Architecture - Foundations and Applications*. Ed. by Uwe Aßmann, Mehmet Aksit, and Arend Rensink. Vol. 3599. Lecture Notes in Computer Science. Berlin, Heidelberg: Springer, 2005, pp. 17–32.

[3] Alexander C. Bock and Ulrich Frank. "Low-Code Platform". In: *Business & Information Systems Engineering* 63.6 (2021), pp. 733–740.

[4] Nicholas Arthur Bzowski. "A Model Driven Architecture Transformation from Ontological Enterprise Models to Low-Code". Diploma Thesis. Vienna, Austria: TU Wien, 2025.

[5] Alessandro Colantoni et al. "Leveraging Model-Driven Technologies for JSON Artefacts: The Shipyard Case Study". In: *ACM/IEEE 24th International Conference on Model Driven Engineering Languages and Systems (MODELS)*. Fukuoka, Japan: IEEE, 2021, pp. 250–260.

[6] Jan L. G. Dietz and Hans B. F. Mulder. *Enterprise Ontology: A Human-Centric Approach to Understanding the Essence of Organisation*. Cham: Springer, 2020.

[7] Alan R. Hevner et al. "Design Science in Information Systems Research". In: *MIS Quarterly* 28.1 (2004), pp. 75–105.

[8] International Organization for Standardization. *Systems and Software Engineering - Systems and Software Quality Requirements and Evaluation (SQuaRE) - System and Software Quality Models*. Geneva, 2011.

[9] Frédéric Jouault and Ivan Kurtev. "Transforming Models with ATL". In: *Satellite Events at the MoDELS 2005 Conference*. Ed. by Jean-Michel Bruel. Vol. 3844. Lecture Notes in Computer Science. Berlin, Heidelberg: Springer, 2006, pp. 128–138.

[10] Barbara Kitchenham and Stuart Charters. *Guidelines for Performing Systematic Literature Reviews in Software Engineering*. Tech. rep. Keele University and Durham University Joint Report, 2007.

[11] Marien R. Krouwel, Martin Op 't Land, and Henderik A. Proper. "From Enterprise Models to Low-Code Applications: Mapping DEMO to Mendix; Illustrated in the Social Housing Domain". In: *Software and Systems Modeling* 23.4 (2024), pp. 837–864.

[12] Odd Ivar Lindland, Guttorm Sindre, and Arne Sølvberg. "Understanding Quality in Conceptual Modeling". In: *IEEE Software* 11.2 (1994), pp. 42–49.

[13] Mendix. *Mendix Platform SDK Documentation*. 2026. URL: https://docs.mendix.com/apidocs-mxsdk/mxsdk/ (visited on 04/10/2026).

[14] Daniel L. Moody. "Theoretical and Practical Issues in Evaluating the Quality of Conceptual Models: Current State and Future Directions". In: *Data & Knowledge Engineering* 55 (2005), pp. 243–276.

[15] Sam Newman. *Building Microservices: Designing Fine-Grained Systems*. 1st ed. Sebastopol, CA: O'Reilly Media, 2015.

[16] Object Management Group. *Meta Object Facility (MOF) 2.0 Query/View/Transformation Specification, Version 1.3*. 2016. URL: https://www.omg.org/spec/QVT/1.3 (visited on 03/23/2026).

[17] Object Management Group. *Model Driven Architecture (MDA) Guide Rev. 2.0*. 2014. URL: https://www.omg.org/cgi-bin/doc?ormsc/14-06-01 (visited on 03/23/2026).

[18] Ken Peffers et al. "A Design Science Research Methodology for Information Systems Research". In: *Journal of Management Information Systems* 24.3 (2007), pp. 45–77.

[19] Klaus Pohl. *Requirements Engineering: An Overview*. Tech. rep. Aachen, Germany: Informatik V, RWTH Aachen, 1996.

[20] Apurvanand Sahay et al. "Supporting the Understanding and Comparison of Low-Code Development Platforms". In: *46th Euromicro Conference on Software Engineering and Advanced Applications (SEAA)*. Porto, Portugal: IEEE, 2020, pp. 171–178.

[21] Monique Snoeck. *Enterprise Information Systems Engineering: The MERODE Approach*. The Enterprise Engineering Series. Cham: Springer, 2014.

[22] Dave Steinberg et al. *EMF: Eclipse Modeling Framework*. 2nd ed. Upper Saddle River, NJ: Addison-Wesley, 2009.

[23] Nguyen Hoang Thuan, Andreas Drechsler, and Pedro Antunes. "Construction of Design Science Research Questions". In: *Communications of the Association for Information Systems* (2018). Accepted Manuscript.

[24] Roel J. Wieringa. *Design Science Methodology for Information Systems and Software Engineering*. Berlin, Heidelberg: Springer, 2014.

[25] Stefan Winkler and Jens von Pilgrim. "A Survey of Traceability in Requirements Engineering and Model-Driven Development". In: *Software and Systems Modeling* 9.4 (2010), pp. 529–565.

[26] Robert K. Yin. *Case Study Research: Design and Methods*. 5th. Thousand Oaks, CA: SAGE Publications, 2014.